Python Lists, Sets, & Tuples

Data with Dylan | Guided Notes | Python Fundamentals #4

Name: _____

Date: _____

☑ HOW TO USE THESE NOTES: Watch the video and fill in the blanks as Dylan explains each concept. Use the writing boxes to capture your own notes and examples. Complete the practice problem at the end!

Section 1: Introduction to Collection Data Types

Topic: Storing multiple values in one variable

What are collection data types?

Complete the definition:

A collection data type stores multiple _____ inside one variable.

The three collection data types covered in this lesson are:

1. _____
2. _____
3. _____

Like other Python variables, collections can store different _____ in the same variable.

Key vocabulary

Complete the definitions:

Mutable means a collection can be _____ after it is created.

Immutable means a collection _____ be changed after it is created.

Ordered means each item has a meaningful _____ .

Unordered means items do not have a fixed _____ .

In your own words - why might a programmer want to store several values in one variable?

Section 2: Lists

Topic: Ordered and mutable collections

What is a list?

Complete the definition:

A list is an _____ and _____ sequence that contains multiple items in one variable.

Lists are created using _____ brackets.

Lists can store different _____ in the same collection.

```
stuff = [6, "hello", True, 3.5]
```

Indexing lists

Because lists are ordered, Python can access items using _____ indexing.

The first item in a list has an index of _____ .

Complete the code below to access the first item:

```
stuff = [6, "hello", True, 3.5]

print(stuff[_______])
```

What value will the code display? _____

Adding to a list

Lists are mutable, so you can add, remove, or change items after creating them.

```
stuff.append("hello world")
```

The `.append()` method adds a new item to the _____ of a list.

`.append() is an` _____ method, which means it directly changes the original list.

Circle the VALID list initialization:

Choice A	**Choice B**	**Choice C**
`fruits = ["apples", "bananas"]`	`fruits = ("apples", "bananas")`	`fruits = {"apples", "bananas"}`

In your own words - why is list indexing useful?

Section 3: Sets

Topic: Storing unique values without a fixed order

What is a set?

Complete the definition:

A set is a _____ collection that stores multiple items in one variable.

Sets are created using _____ braces.

Sets are _____ , which means you can add or remove items from the set.

Sets are _____ , so their items do not have fixed positions.

Sets do not allow _____ values.

```
fruits = {"apples", "bananas", "oranges"}
```

Understanding set behavior

Each time you display a set, its items may appear in a different _____ .

Why? Because sets are _____ .

This code will not work because a set does not use _____ .

```
fruits[0]
```

Duplicate values in sets

How many "apples" values remain in this set? _____

```
fruits = {"apples", "apples", "bananas"}
```

Python treats _____ and _____ as equivalent values inside a set.

In your own words - what happens when you pass a list with duplicate values into a set?

```
fruit_list = ["apples", "apples", "bananas", "bananas"]
fruit_set = set(fruit_list)
```

Circle the VALID set initialization:

Choice A	Choice B	Choice C
`ids = ["A101", "A205"]`	`ids = {"A101", "A205"}`	`ids = ("A101", "A205")`

Section 4: Tuples

Topic: Ordered collections that cannot be changed

What is a tuple?

Complete the definition:

A tuple is an _____ sequence that stores multiple values in one variable.

Tuples are created using _____ .

Like lists, tuples are _____ and allow duplicate values.

Unlike lists, tuples are _____ .

```
stuff = ("apples", "hello", True, 5, 5)
```

Indexing tuples

Because tuples are ordered, you can use _____ indexing.

Complete the code below to access "hello":

```
stuff = ("apples", "hello", True, 5, 5)

print(stuff[_______])
```

Changing a tuple

What happens when you try to replace a tuple item?

```
stuff[1] = "bye"
```

Python raises a _____ . Why does this happen?

Creating an updated tuple

```
original_tuple = (1, 2, 3)
new_tuple = (4, 5)

updated_tuple = original_tuple + new_tuple
```

The `+` operator creates a _____ tuple. The result must be _____ to a new variable.

Circle the VALID tuple initialization:

Choice A	**Choice B**	**Choice C**
`coordinates = {"x", "y"}`	`coordinates = ("x", "y")`	`coordinates = ["x", "y"]`

In your own words - why are tuples useful for coordinates that should not change?

Section 5: Collection Operations Reference

Topic: Reading, adding, counting, and combining collection values

Fill in the missing method or operation names as you review each example.

CODE	METHOD / OPERATION NAME	RESULT / NOTES
`stuff[0]`	_____	Accesses the first item in an ordered list or tuple.
`stuff.append("hello world")`	_____	Adds a new item directly to the end of a list.
`fruits.add("mango")`	_____	Adds one new item to a set.
`len(fruits)`	_____	Returns the number of items. Duplicate set values count only once.
`updated_tuple = original_tuple + new_tuple`	_____	Combines tuple values and stores the result in a new tuple variable.

Compare the outcomes

len({"apples", "apples", "apples"}) returns _____ .

A list or tuple can use square brackets for _____ .

A set cannot use square brackets with an index because its items are _____ .

In your own words - when would `.append()` and `.add()` be useful?

Section 6: Choosing the Right Collection Type

Topic: Matching collection behavior to a programming task

For each situation, choose the best collection type.

PROGRAM NEED	BEST COLLECTION TYPE	WHY?
Store a playlist where order matters and songs may repeat.	_____	_____
Store unique user tracking IDs with no duplicate values.	_____	_____
`Store a fixed (x, y) coordinate that should not change.`	_____	_____
Store several values that need to be changed later.	_____	_____

Complete the comparison

A list is best when you need an _____ collection that can be changed.

A set is best when you need _____ values and order does not matter.

A tuple is best when you need an ordered collection that should remain _____ .

In your own words - what is the biggest difference between a list and a tuple?

In your own words - what is the biggest difference between a list and a set?

Section 7: Putting It All Together

Topic: Lists, sets, and tuples reference table

Using what you have learned, complete the full collection reference table:

COLLECTION TYPE	MUTABLE? (YES/NO)	ORDERED? (YES/NO)	ALLOWS DUPLICATES? (YES/NO)	YOUR OWN EXAMPLE
List	Yes	Yes	Yes	
Set	Yes	No	No	
Tuple	No	Yes	Yes	

Section 8: Quick Recap

Test yourself - try to complete these without looking back!

1. A list is an _____ and _____ collection.
2. List and tuple indexing starts at _____.
3. Sets are _____, so their values do not have fixed positions.
4. Sets do not allow _____ values.
5. Tuples are _____, so changing an item raises a _____.

Practice Problem

⚡ **Challenge - Pause the video and try this yourself before Dylan reveals the answer!**

```
user_ids = ["U-101", "U-205", "U-101", "U-310"]
coordinates = (47.61, -122.33)
```

Goal: Create a new collection called `unique_ids` that stores each user tracking ID only once.

Why? Why is a set the best collection type for `unique_ids`?

Why? Why should `coordinates` remain a tuple instead of becoming a list?

Write your full statement below:

```
unique_ids =
```

How did it go? What part tripped you up?

Personal Notes

Use this space to capture any additional observations, questions, or ideas from the video.

Key Terms Reference

Use this table to build your Python vocabulary. Write definitions in your own words.

TERM	DEFINITION IN YOUR OWN WORDS
Collection Data Type	
Data Structure	
List	
Set	
Tuple	
Sequence	
Mutable	
Immutable	
Ordered	
Unordered	
Indexing	
Zero-Based Indexing	
Duplicates	
`Square Brackets []`	

Key Terms Reference (continued)

Continue building your Python vocabulary. Write definitions in your own words.

TERM	DEFINITION IN YOUR OWN WORDS
`Parentheses ()`	
Curly Braces {}	
`append()`	
`add()`	
`len()`	
`set()`	
In-Place Method	
`Addition Operator (+)`	
Reassignment	
TypeError	